
Detecting Hidden ML Training With Zero-Overhead Telemetry

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Hardware-enabled monitoring of GPU workloads underpins many proposals for
2 AI compute governance, but if developers can defeat monitoring mechanisms, such
3 schemes are unworkable. We evaluate the adversarial robustness of GPU work-
4 load classification using only zero-overhead, privacy-preserving NVML telemetry:
5 content-agnostic signals that observe physical effects of computation without ac-
6 cessing model weights, training data, or hyperparameters. Across 5 rounds of
7 monitor-evader iteration, we evaluate 20 evasion strategy families on 9 GPU mod-
8 els spanning 4 architecture generations. We develop a classifier that achieves 98.2%
9 binary accuracy at identifying training workloads across the whole corpus, and
10 43–87% accuracy against the most challenging unexpected workloads even when
11 they are adversarially disguised.

12 1 Introduction

13 Compute governance proposals such as [Shavit, 2023, Baker et al., 2025, Scher et al., 2025] call
14 for regulations including evaluations, audits, and registration or prohibition of training runs above
15 specified compute thresholds [Heim and Koessler, 2024]. US Executive Order 14110, in effect from
16 October 2023 until rescinded in January 2025, previously required developers who perform training
17 runs exceeding 10^{26} floating-point operations (FLOP) to report those runs to the federal government,
18 along with their cybersecurity and risk management practices [The White House, 2023].

19 However, the ability to regulate ML training compute requires that regulators are able to know when
20 training occurs, and that developers cannot discreetly run training workloads undetected. Work is
21 underway to establish the feasibility of geolocation of computing hardware [Brass and Aarne, 2024]
22 and registries of AI chips [Fist and Grunewald, 2023]. But registration and geolocation are insufficient
23 unless the regulator understands what the chips are doing.

24 Past work demonstrates the feasibility of distinguishing ML from non-ML computing when the
25 adversary is not trying to disguise the workloads [Tang et al., 2022, Weakley et al., 2025], and
26 existing governance literature discusses masking or structuring as possible evasion tactics [Heim
27 et al., 2024, Seferis and Fist, 2025]. However, there has not yet been a systematic effort to iterate
28 on this cat-and-mouse dynamic and determine whether the problem landscape favors detection or
29 evasion.

30 Therefore, we investigate the adversarial robustness of GPU workload classifiers, trained on signals
31 extracted from built-in telemetry readings. We build the classifier as a prerequisite, then extensively
32 red-team it with 20+ categories of real and adversarial workloads across 100+ GPU-hours of ML
33 experiments.

34 We find that zero-overhead GPU telemetry is sufficient to classify workloads as training vs non-
35 training with 98.2% accuracy across 9 GPU types spanning consumer and datacenter hardware

(Ampere through Blackwell). Crucially, this telemetry is content-agnostic and privacy-preserving: it observes physical effects of computation (power, temperature, utilization) without accessing model weights, training data, or hyperparameters [Shavit, 2023, O’Gara et al., 2025].

Through 5 rounds of monitor-evader iteration, we implement 20 families of evasion strategies, including 4 multi-GPU model- and/or data-parallel configurations on nodes of up to 8 datacenter cards. Leave-one-strategy-out evaluation — training classifiers on some evasion tactics while evaluating on unknown workloads — shows mean accuracy of 95.2% on unseen non-white-box strategies (92.2% including Round 5 white-box attacks), with all multi-GPU workloads detected at $\geq 99.6\%$ and only 5 of 20 families under 90% detection against the best classifier. Physics-based evasion-resistant features (cumulative energy, power-memory correlation, pre-allocation memory-plateau features) provide the signal needed to catch the hardest remaining single-GPU strategies.

We release a privacy-preserving, content-agnostic classifier achieving 98.2% binary (training-vs-not) accuracy on a corpus spanning training, inference, and non-ML workloads run on NVIDIA hardware in generations Ampere through Blackwell, using only zero-overhead NVIDIA Management Library (NVML) telemetry signals. We evaluate cross-architecture generalization, demonstrating that classifiers trained on newer hardware detect training on older hardware without re-fitting. Source code for the non-ML, inference and training, and adversarial evasion workloads is available in the project repository. We invite the AI governance and ML research communities to red-team our classifier and publish new evasion strategies that might defeat it, which will make it even more robust in the future.

2 Background and Threat Model

2.1 Telemetry Signals

Our classifier operates on nine always-available NVML counters (GPU utilization, memory utilization, memory used, power draw, temperature, SM clock, memory clock, PCIe TX/RX) sampled at 1 Hz. These counters are available on every NVIDIA GPU from Pascal onwards with no runtime overhead: polling them does not measurably burden or slow down the monitored workload, and does not require any additional configuration. The readings do not involve access to application code, model weights, or training data — these signals are purely physical or electrical measurements of utilization of different components of the GPU.

In contrast, kernel-level profiling via Nsight Compute reads Performance Monitoring Counters at 1200–5300% overhead [Xu et al., 2026]. With datacenters already using huge and rapidly increasing amounts of energy [You et al., 2025] it is more sustainable and inexpensive to add as little overhead as possible. This makes our result a lower bound: anything that works with this minimal observational budget also works with more information, so a real deployment monitor that can access more telemetry or physical channels starts from this demonstrated detection level and only becomes stronger.

2.2 Threat Model

We consider two types of evaders. One is a cloud tenant: a user or AI developer renting GPU time, attempting to disguise ML training as inference or non-ML workloads to evade the cloud provider’s monitoring. Proposals such as Heim et al. [2024] rely on the intermediary role of compute providers acting as “record keepers, verifiers, and, in some cases, even enforcers” of AI governance. In this case, the adversary controls user-space software but not the host OS, hypervisor, or hardware. One type of evasion they might attempt is *compute structuring*, wherein training runs are split across multiple cloud providers to dodge a reporting threshold. Seferis and Fist [2025] argue that these attacks are detectable, but this relies on the ability of each cloud host to reliably classify the workloads running on its own GPUs. If tenants can disguise training as other workloads and evade recognition by the host, the compute-structuring defense collapses. Our classifier provides the foundation that allows this type of enforcement to work, and our adversarial evaluation is a test of its robustness.

Another type of evader is a hardware operator. They own or operate the hardware, typically in a datacenter facility, and attempt to hide unauthorized AI training from an external regulator. The adversary controls the full software stack and can modify workload scheduling. In order to prevent this evader from defeating the monitoring scheme, we need to equip their chips with a *hardware-enabled governance mechanism* (HEM) that ensures that the evader cannot (i) disable NVML polling or tamper with the returned counter values at the hardware level, nor (ii) inject synthetic telemetry

into the monitor’s ingestion channel. Both assumptions describe mechanisms the software stack alone cannot enforce; they are the minimum hardware-enabled-mechanism (HEM) guarantees our results depend on [Aarne et al., 2024, O’Gara et al., 2025]. Section 5.5 discusses the HEM requirements in more detail as a limitation and as a direction for development.

The hardware operator is theoretically more challenging to monitor than the cloud tenant, but both evaders are subject to a fundamental physical constraint: a backward pass requires $\sim 2\times$ the FLOPs of a forward pass, and therefore flips more transistors and consumes more energy. Evasion strategies designed against one category of evader transfer reasonably well to the other, because one major constraint is the physical energy requirement, not specific software.

2.3 Classification Task

Our dataset contains telemetry records from our 106 distinct ML training workloads (a mix of ordinary ML tasks and adversarial workloads disguised as inference or non-ML), 40 distinct ML inference workloads, and 19 non-ML workloads. Our classifier can distinguish these with high accuracy and precision, as described in Methodology and Results below. But the fundamental governance-relevant question is binary: **Is a monitored GPU being used for training?** We tested several options before selecting a random forest classifier, which exhibited the best performance on this task. Details are provided in Appendix B.

2.4 Related Work

The field of compute governance contains many proposals for regulations that implicitly rely on the ability to reliably identify when developers are using compute for AI training, and many proposals for hardware-enabled mechanisms that assist our classifier in securely providing that information. We extend the existing research relevant to workload classification, and past work covering adjacent tasks such as identifying inference workloads, detecting cryptocurrency mining, and detecting training workloads with physical side-channel measurements or software methods with runtime overhead. Our classifier identifies training workloads using zero-overhead software telemetry. For more detailed background information, see Appendix A.

3 Methodology

3.1 Data Collection

3.1.1 Hardware

Hardware used for this project includes 9 models of NVIDIA GPUs across 4 architecture generations. Consumer hardware is generally not covered by proposed AI governance and verification [Heim et al., 2024, Scher et al., 2025], but was included to test generalization across different hardware.

Table 1: Instances used in this research by type and quantity of GPUs. Some experiments involved fewer than the maximum number of GPUs available on the instance.

GPU Model	Architecture	VRAM	Context	GPUs
RTX 4050 Laptop	Ada Lovelace	6 GB	Local laptop	1
RTX 5080	Blackwell	16 GB	Local desktop	1
RTX 5090	Blackwell	32 GB	Cloud	1, 2
A100 SXM4 40GB	Ampere	40 GB	Cloud	1, 8
A100 SXM4 80GB	Ampere	80 GB	Cloud	1, 8
H100 80GB	Hopper	80 GB	Cloud	1, 8
H100 NVL	Hopper	94 GB	Cloud	1, 2
H200 / H200 NVL	Hopper	141 GB	Cloud	1, 2, 8
B200	Blackwell	192 GB	Cloud	1, 2, 8

120 3.1.2 Workloads

121 We conducted 1404 runs of 162 distinct workloads, totaling ~ 117 GPU-hours (~ 65 wall-clock hours)
 122 of experiments, for a total of 445,310 telemetry readings $\times$ 9 signals per reading. Not every workload
 123 is run on every hardware configuration, for instance due to memory limitations.

Table 2: Workload categories collected in this study.

Category	Workloads	Runs
ML training	106	867
ML inference	40	352
Other	16	185

124 We conducted 867 runs of ML training, including 180 of supervised training (e.g., ResNet on
 125 CIFAR-10, GPT-2 on WikiText, BERT fine-tuning), 72 of distributed data-parallel training, 152 of
 126 fully-sharded data-parallel training on models with 7B to 70B parameters, and 42 of LoRA fine-tuning.
 127 This also included 421 runs of adversarial evasion (369 non-white-box plus 52 Round 5 white-box),
 128 testing strategies such as utilization modulation, memory minimization or preallocation, clock
 129 throttling, interleaving training with inference or non-ML workloads, mimicking FFT computations
 130 or cryptocurrency mining, and padding with idle time.

131 For inference, we did 352 runs, including 49 of image classification, 264 of LLM inference (TinyL-
 132 lama, Phi-3, Qwen-2.5 3B through 72B, Mistral and Mixtral, DeepSeek-V3 and R1), and 39 of other
 133 modalities (e.g., Whisper speech-to-text, SDXL text-to-image diffusion).

134 We also did 185 runs of non-ML workloads, including 65 of scientific HPC (e.g., fast Fourier
 135 transform, n-body simulation), 36 of cryptocurrency mining, 36 of rendering or video processing
 136 (Blender, rendering, FFMPEG), 34 idle-GPU baselines, and 14 miscellaneous additional non-ML
 137 runs.

138 A full catalog of adversarial workloads is provided in Appendix C and source code for every workload
 139 is available in the project repository.

140 3.1.3 Telemetry

141 During all workloads, we sample 9 raw NVML signals at 1 Hz: GPU utilization (%), memory
 142 utilization (%), memory used (MB), power draw (W), temperature ($^{\circ}\text{C}$), SM clock (MHz), memory
 143 clock (MHz), PCIe TX bandwidth (MB/s), and PCIe RX bandwidth (MB/s).

144 Additionally, for 49 workloads tested on the local RTX 5080 desktop, we validate the telemetry’s
 145 reported power draw using an external Tapo P115 smart plug reading the system’s power draw with
 146 timestamps matching within 0.5s.

147 We extract features from the raw signals as described in Appendix B.3.

148 3.2 Classifier

149 We selected the best-performing of several candidate models. RandomForest [Breiman, 2001] with
 150 400 trees and no maximum depth was the most accurate, and XGBoost [Chen and Guestrin, 2016]
 151 was a close runner-up. We use RF for all reported headline results but retain XGBoost for gain-
 152 based feature-importance analysis. Alternative classifiers considered include SVM-RBF and logistic
 153 regression. Classifier comparison is provided in Appendix B.

154 The feature set is designed along two signal layers: (1) a structural, roofline [Williams et al., 2009]
 155 layer that identifies workloads occupying a training-like operating region in arithmetic-intensity
 156 space (mean power, mean utilization), and (2) a temporal telemetry layer that resolves ambiguity
 157 within that region using time-domain features (autocorrelation, CV, periodicity). Both stages of the
 158 two-stage pipeline use the same underlying feature vocabulary: stage 1 retains the full set; stage 2
 159 drops the level features to reduce overfitting and prevent exploitation by adversarial strategies that
 160 mimic specific usage levels.

3.3 Adversarial Evaluation Protocol

Our refinement procedure iterates on the classifier and the evader through five rounds of co-evolution. In each round, we design new evasion strategies for the attacker, targeting the classifier’s existing weaknesses, and then the defender trains a hardened classifier incorporating the new adversarial examples. Qualitatively, the classifier improves through stages which can be described as *baseline* — trained only on non-adversarial data consisting of real-life workloads — to *hardened* — defeating initial adversarial strategies introduced in earlier rounds — to *evasion-resistant* — hardened against more sophisticated obfuscation, plus sensing physics-based features that capture fundamental computational invariants that the adversary cannot suppress without aborting training.

Table 3: Summary of adversarial iteration over several rounds of increasing evader and classifier sophistication. Workload labels are in parentheses; a full catalog is provided in Appendix C.

Round	Attacker introduces	Defender response
1	Utilization manipulation: modulation (A), low-utilization training (B), temporal disruptions (D)	Retrain → hardened classifier
2	Memory reduction and non-ML mimicry: minimal-memory training (E), interleaving (F), clock throttling (G), FFT/mining mimicry (H), stochastic (I), PID control (J)	Retrain → hardened (improved)
3	Training-as-inference: online learning (K), training diluted with $N \in \{2, 5, 10, 20\} \times$ inference (L)	Add power-memory correlation + cumulative energy features → evasion-resistant
4	Composite evasion: min-memory composite strategy (M), gradient accumulation structuring (N), composite idle-padding sweep (0–95% idle fraction); multi-GPU distributed data-parallel variants (K_ddp, K_ddp_accum, L_ddp, L_ddp_stagger)	Retrain on multi-GPU data → evasion-resistant (improved)
5	White-box (attacker given classifier internals): full-model dilution and oracle-driven evasion (<i>whitebox_full</i>), and LoRA + dilution (<i>whitebox_lora</i>)	Add pre-allocation memory-plateau features (<i>first_30s_mem_delta</i> , <i>time_to_mem_plateau</i>); collect more non-adversarial examples of matched-size dense LLM inference on datacenter hardware

This iteration process yielded 20 families of evasion strategies covering five technique classes: utilization manipulation, memory reduction, workload mimicry, training-as-inference (e.g., dilution or mixing the two), multi-GPU evasion (e.g., structuring workloads across multiple GPUs to look less like training on some of them individually), and white-box adversary (examining previous classifiers to discover niche weaknesses).

For each strategy family, we train the classifier on real (non-adversarial) training, inference, and non-ML workload data, plus every *other* adversarial strategy, and test on the held-out family. This tests the classifier’s ability to generalize to unseen, novel evasion techniques, which informs us about the real-world governance case where an adversary invents a strategy we have not yet catalogued.

4 Results

4.1 Baseline Classification

First we present the non-adversarial, pre-hardening baseline. We replicate the previous finding that the classification problem is solvable and real training and inference workloads can be distinguished from non-ML workloads [Tang et al., 2022, Xu et al., 2026] such as cryptocurrency mining and graphics rendering.

Pooling all 9 GPU models, the classifier achieves $98.21\% \pm 0.84\%$ binary (training-vs-not) accuracy with macro-F1 = 0.982.¹ Latency is quite quick, with 98.15% at 5s windows and 98.21% at 30s. Analysis of detection latency is provided in Appendix F.

¹ RandomForest, 30 s windows, stride 15s, 5-fold grouped CV, 26,496 windows across 1,396 runs that produced at least 5 NVML samples. See Appendix B for the full classifier comparison.

188 Accuracy was evaluated with the data split by GPU model, comparing single-GPU training against
 189 pooled training across all 9 GPUs:

Table 4: Per-GPU classification accuracy. The Single-GPU CV column trains and tests on different workloads from the same GPU under 5-fold cross-validation. The LOGO (leave-one-GPU-out) column trains on data from the *other* 8 GPUs and tests on the held-out GPU, the deployment scenario when encountering hardware not in the training set. In-distribution pooled CV, training on workloads from *all* GPUs, reaches 100% on every GPU.

GPU	Single-GPU CV	LOGO
A100-SXM4 40GB	100.00%	100.00%
A100-SXM4 80GB	100.00%	100.00%
B200	99.55%	99.43%
H100 NVL	99.52%	100.00%
H200	99.73%	100.00%
H100 80GB HBM3	99.43%	100.00%
RTX 5090	97.57%	100.00%
RTX 5080	99.93%	100.00%
RTX 4050 Laptop	95.14%	100.00%

190 In-distribution pooled classification is essentially solved: when data from all 9 GPUs is used for
 191 5-fold grouped CV, every GPU’s held-out folds reach 100% accuracy. The harder, governance-
 192 relevant question is leave-one-GPU-out (LOGO) — train on the other 8 GPUs, test on the held-out
 193 model — which is the regulator’s deployment scenario when evaluating telemetry from a new type of
 194 hardware not observed before. Figure 1’s bottom row and Table 4’s LOGO column show that this
 195 also generalizes well: the classifier reaches 100% on 8 of 9 held-out GPUs and 99.4% on B200, even
 196 with no training data from the target hardware. Cross-GPU transfer when training on a *single* GPU
 197 (the upper rows of Figure 1) is weaker: accuracy is worse on other hardware, particularly between
 198 consumer and datacenter hardware. But a regulator with access to telemetry from a variety of GPUs
 199 can train one classifier and deploy it site-agnostically, even to GPU models they have not yet sampled.

200 4.2 Adversary-Defender Iteration

201 Table 5 shows the key results.

202 The first row shows baseline classification performance on only clean, non-adversarial workloads.
 203 The baseline classifier, trained on all but one workload at a time, achieves an average of 97.6%
 204 accuracy on the other, held-out workload.

205 In Round 1, the evader introduces workload obfuscation strategies revolving around utilization
 206 manipulation. The baseline classifier achieves 6–96% binary accuracy at classifying workloads using
 207 these strategies as training vs not-training. Then the defender iterates on these strategies, achieving
 208 100% in-distribution accuracy at classifying these workloads after it has seen them, or 59–100%
 209 accuracy at classifying unseen strategies from Round 1 after learning from the other strategies.

210 In Round 2, the evader introduces memory reduction and mimicry strategies, and in Round 3,
 211 the evader introduces so-called “training-as-inference” strategies, camouflaging online learning as
 212 inference or diluting gradient updates between many batches of inference. Each time, the evader’s
 213 new strategies achieve partial success against the baseline classifier and the previous round’s classifier,
 214 and each time, the defender trains the new classifier to always detect the new evasive workload if it
 215 has seen that strategy, or detect them the majority of the time if it has seen strategies from that round
 216 but not the specific strategy it is tested on.

217 In Round 4, we introduced new evasion strategies for disguising single-GPU training, as well as
 218 strategies for disguising multi-GPU training which attempt to prevent the detection of training on
 219 each GPU’s telemetry by utilizing other GPUs to shift the workload around. Round 4’s strategies
 220 achieved some initial evasion against the classifier from Round 3, but were very thoroughly rendered
 221 ineffective once the classifier was hardened against them.

222 Finally, in Round 5, we gave the evader white-box access to the classifier developed in previous
 223 rounds. The evader was able to examine the internal mechanics of the classifier to find precise exploits.

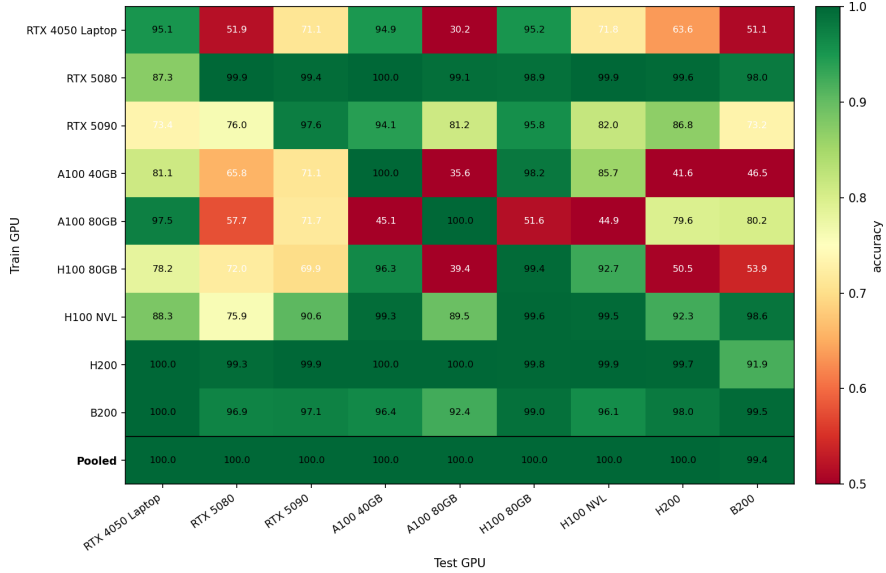


Figure 1: Cross-GPU transfer heatmap (binary RF, 30 s windows). Cell values are accuracy when training on the row GPU and testing on the column GPU. Datacenter-to-datacenter transfer is consistently $\geq 90\%$. Cross-consumer and consumer $\leftrightarrow$ datacenter transfer is weaker: RTX 5080/5090/4050 rows drop below 50% on many datacenter targets. The bottom row is leave-one-GPU-out (LOGO): for each test GPU, the classifier is trained only on data from the other 8 GPUs. By contrast, in-distribution pooled CV (with the held-out GPU’s runs *included* in training) reaches 100% on every GPU. LOGO measures the harder question of generalization to unseen hardware.

For example, while red-teaming the Round 4 classifier, we noted that the classifier partly relies on features derived from windows 5s or longer. Therefore, one prospective evader strategy is to insert 4s of training across the boundary of two 5s windows, spending 40% of 10s on training but taking up less than half of the time of each window, misleading the classifier towards identifying this as the other workload that was running. Another tactic was to use a copy of the Round 4 classifier against the defender in real-time, running training as long as the defender believed $P(\text{training}) < 0.30$ and temporarily replacing training with another workload whenever the defender became suspicious. These tactics achieved up to 94% evasion against the Round 4 classifier. However, once the defender was able to train against them, they were rendered ineffective, with (as usual) 100% detection of seen evasion strategies, and notably 43–87% detection of unseen white-box strategies after hardening against other Round 5 strategies.

Round 5 represents a disaster scenario for governance, equivalent to a developer infiltrating the governing agency, stealing the classifier, and orchestrating their workload in a way that is designed to be maximally undetectable by the classifier’s exact algorithm.² However, once a classifier is hardened against a few such exploits, other white-box attacks are much less effective in the future.

5 Discussion

5.1 An Involuntary Witness

During ML training, a backward pass (computing gradients at each parameter) involves twice as much computation as the forward pass [Kaplan et al., 2020]. This imposes a fundamental constraint on training: for the same number of tokens processed, it is three times as energy-intensive as inference, so you can only either shuffle energy consumption around or reduce the amount of training conducted, but ultimately you cannot make them use the same amount of energy without eliminating the step of updating the weights, which would leave the workload unable to train the model.

²In an absolute worst-case scenario, the developer could infiltrate the regulator, replace the classifier with a compromised version containing a backdoor, and also compromise the monitored hardware.

Table 5: Arms race summary showing each classifier state’s detection rate against each round’s evasion strategies. Baseline classifiers are trained only on non-adversarial data. In each round, the adversary introduces new strategies to defeat the previous version of the classifier, then the defender teaches the classifier to recognize training disguised with the new evasion strategies. All classifiers use class-balanced training, with class weights adjusted inversely to their number of samples so that extra data collection on one type of workload does not overfit the classifier. *Notes:* The Baseline-row *Hardened, unseen* value (97.6%) refers to the baseline classifier’s accuracy at evaluating held-out non-adversarial workloads when trained on all other non-adversarial workloads. Prior to Round 2, there was no hardened classifier; the “previous round’s classifier” was the baseline trained only on non-adversarial workloads. The Round 5 white-box evader is given access to the evasion-resistant Round 4 classifier; if it had instead targeted the baseline classifier with full white-box access, detection would be very low (likely 0–20%).

Round	Attacker capability	Baseline classifier detection of new evader	Previous round classifier detection of new evader	Hardened detection in-distribution	Hardened detection, unseen workloads
Baseline	(No evasion; real workloads only)	N/A	—	—	97.6%
1	Utilization manipulation (A, B, D)	6–96%	—	100%	59–100%
2	Memory reduction / mimicry (E, F, G, H, I, J)	0–91%	20–95%	100%	80–100%
3	Training-as-inference (K online, L diluted)	52–60%	48–57%	100%	87–100%
4 (Single-GPU)	Composite + grad accum (M, N, composite 0–95% idle)	19–100%	83–100%	100%	98–100%
4 (Multi-GPU)	Multi-GPU distributed evasion strategies (K/L DDP + stagger)	72–100%	71–100%	100%	100%
5	White-box (WB-full, WB-LoRA) with R4 classifier internals exposed	N/A	6–100%	100%	43–87%

Our empirical evidence is consistent with this and suggests that power draw is a valuable feature for distinguishing training from inference. Speculatively, as we iterate further on the classifier-evader process in the future, evaders may learn to defeat other features but energy is a fundamental constraint, so energy-based features may rise in importance as the classifier hardens further.

5.2 Single- vs Multi-GPU Detection

Our datacenter experiments (on 8-GPU A100, H200, B200 nodes) test both threat models above. We designed 4 multi-GPU evasion strategies: DDP online learning, DDP diluted training, staggered dilution (spreading NVLink traffic across time), and gradient accumulation across ranks. All are detected at 99.6–100% in leave-one-strategy-out evaluation (Figure 4). Multi-GPU training is harder to hide than the much less important single-GPU case because traffic and interconnect usage patterns that differ between ordinary multi-GPU large-model training and inference provide additional opportunities for the classifier to detect differences in signals, and more that the evader will have to convincingly disguise. For example, our classifier learned to recognize AllGather and ReduceScatter NVLink traffic associated with forward/backward steps in FSDP training at 7B–70B scale and applied this to detect obfuscated multi-GPU training even when the adversary tried to manipulate the communication patterns. Multi-GPU training may create distinctive telemetry signatures on each GPU even when workloads are structured so that an individual GPU shows only moderate utilization.

5.3 Privacy Preservation

The NVML metrics queried by our classifier are content-agnostic: it observes the physical and architectural effects of computation (power draw, temperature, utilization, memory) without accessing model weights, training data, hyperparameters, or application logic. This addresses a central deployment concern in shared cloud settings, where governance mechanisms that require inspecting proprietary information face legal, operational, and ethical objections [Shavit, 2023, O’Gara et al., 2025]. Our classifier demonstrates that the governance-relevant question (“is this GPU training a model?”) can be answered without any access to the semantic content being processed.

5.4 Layered Defense

If telemetry is collected and monitored at multiple layers (device, cluster, facility), an adversary must evade all layers simultaneously [O’Gara et al., 2025]. The evasion cost is cumulative across layers, so our results using NVML telemetry are a lower bound on what a layered monitor can achieve. DCGM counters (tensor-core activity, NVLink throughput) and wall- or rack-level power measurements provide additional signal. If combined with tamper-resistant hardware such as flexHEGs [Petrie and Aarne, 2025], software-level evasion alone could not defeat such a layered monitor.

5.5 Implications for HEMs

Our results naturally suggest a minimal HEM design for preventing covert training on monitored hardware. The software-level classifier demonstrates that always-available NVML counters at 1 Hz carry enough signal to detect training at 98.2% binary accuracy with LOO adversarial robustness — but this result only holds if the counters themselves are trustworthy. A deployable monitoring system therefore requires three HEM components in addition to the classifier:

1. **Tamper-resistant NVML read path.** The monitoring agent must read counter values from a source the adversary cannot tamper with — e.g., an attested enclave on the GPU firmware boundary (as in Aarne et al. [2024]) or a verified read path through secure-boot-chained firmware [Petrie, 2024, O’Gara et al., 2025]. Without this, an adversary with software privilege simply returns forged values.
2. **Authenticated telemetry channel.** Telemetry in transit from the GPU agent to the verifier must be signed with a hardware-rooted key so an in-path adversary cannot substitute a permitted workload’s telemetry for the actual workload. This extends to rate-limiting and nonce-binding to prevent replay of previously-observed benign signatures.
3. **Secure boot of the monitoring software.** The process itself, its classifier weights, and its feature-extraction code must be attested so an adversary cannot replace them with a permissive version (e.g., a classifier that labels everything as inference).

6 Conclusion

Zero-overhead GPU telemetry monitoring is a viable mechanism for compute governance. Defenders are structurally favored, especially in the most relevant case: our classifier is extremely effective at detecting all evasion strategies tested on multi-GPU datacenter nodes, correctly classifying 98–100% of unforeseen workloads in that setting. Single-GPU evasion has slight success, with a 6.2% evasion rate in held-out evaluation, on average across the 14 single-GPU non-white-box strategies. Only one type of evasion, white-box LoRA, achieves under 50% detection rate against the evasion-resistant classifier, even with access to the defender’s previous classifier. The classifier learns to recognize fundamental features that cannot be eliminated without sacrificing the ability to update model weights. The results are robust across four generations of NVIDIA hardware — three datacenter (Ampere, Hopper, Blackwell) and one consumer (Ada Lovelace) — with strong generalization to other GPU models in hold-out validation.

The compute-monitoring proposals of Shavit [2023], Baker et al. [2025], and O’Gara et al. [2025] are empirically realisable against a realistic software-level evasion adversary, conditional on the HEM assumptions of tamper-resistant telemetry, an authenticated telemetry channel, and a secure classifier. With those hardware guarantees in place, the adversary is left with a physical cost trade-off (dilution and idle-padding) whose economics are unviable at frontier scale. By demonstrating that it is possible to construct a classifier that accurately identifies training activity, we advance the progress of governance from the ML problem to the hardware security problem: “Can we build the matching HEM?” We invite researchers and ML experts to red-team our classifier and publish evasion strategies that might defeat it, so that we can iterate on any such findings to ultimately strengthen its robustness.

References

- O. Aarne, T. Fist, and C. Withers. Secure, governable chips: Using on-chip mechanisms to manage national security risks from AI & advanced computing. Technical report, Center for a New Amer-

ican Security (CNAS), Jan. 2024. URL <https://www.cnas.org/publications/reports/secure-governable-chips>.

G. Almusaddar, Y. Zhang, S. Ganjisaffar, B. Williams, Y. D. Liu, D. Ponomarev, and N. Abu-Ghazaleh. ShadowScope: GPU monitoring and validation via composable side channel signals, 2025. URL <https://arxiv.org/abs/2509.00300>. To appear in ASPLOS 2026.

Anonymous. Differential architecture. Submitted to the 53rd Annual International Symposium on Computer Architecture (ISCA 2026); under review. Author list withheld during anonymous review., 2026.

S. E. Arefin and A. Serwadda. Exploiting HDMI and USB ports for GPU side-channel insights. In *Proceedings of the 15th ACM Conference on Data and Application Security and Privacy (CODASPY)*. ACM, 2025. doi: 10.1145/3714393.3726511.

M. Baker, G. Kulp, O. Marks, M. Brundage, and L. Heim. Verifying international agreements on AI: Six layers of verification for rules on large-scale AI development and deployment, 2025. URL <https://arxiv.org/abs/2507.15916>. Also published as RAND Working Paper WR-A4077-1.

A. Brass and O. Aarne. Location verification for AI chips. Technical report, Institute for AI Policy and Strategy (IAPS), 2024. URL <https://www.iaps.ai/research/location-verification-for-ai-chips>.

L. Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001. doi: 10.1023/A:1010933404324.

T. Chen and C. Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 785–794. ACM, 2016. doi: 10.1145/2939672.2939785.

Z. Chen, S. W. D. Chien, P. Qian, and N. Zilberman. Detecting anomalies in machine learning infrastructure via hardware telemetry, 2025. URL <https://arxiv.org/abs/2510.26008>.

G. Esposito, J.-D. Guerrero-Balaguera, J. E. Rodriguez Condia, M. Sonza Reorda, M. Barbiero, and R. Fortuna. GPU under pressure: Estimating application’s stress via telemetry and performance counters, 2025. URL <https://arxiv.org/abs/2511.05067>.

T. Fist and E. Grunewald. Preventing AI chip smuggling to China. Technical report, Center for a New American Security (CNAS), Oct. 2023. URL <https://www.cnas.org/publications/reports/preventing-ai-chip-smuggling-to-china>.

A. Gangwal, S. G. Piazzetta, G. Lain, and M. Conti. Detecting covert cryptomining using HPC, 2019. URL <https://arxiv.org/abs/1909.00268>. Also published in CANS 2020.

L. Heim and L. Koessler. Training compute thresholds: Features and functions in AI regulation, 2024. URL <https://arxiv.org/abs/2405.10799>.

L. Heim, T. Fist, J. Egan, S. Huang, S. Zekany, R. Trager, M. A. Osborne, and N. Zilberman. Governing through the cloud: The intermediary role of compute providers in AI regulation. Technical report, Centre for the Governance of AI (GovAI), 2024. URL <https://www.governance.ai/research-paper/governing-through-the-cloud>.

J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei. Scaling laws for neural language models, 2020. URL <https://arxiv.org/abs/2001.08361>.

G. Kulp, M. Siu, and L. Heim. Insights from a workshop on hardware-enabled governance mechanisms. Technical Report CF-A3056-1, RAND Corporation, 2025. URL https://www.rand.org/pubs/conf_proceedings/CFA3056-1.html.

I. Latif, A. C. Newkirk, M. R. Carbone, A. Munir, Y. Lin, J. Koomey, X. Yu, and Z. Dong. Empirical measurements of AI training power demand on a GPU-accelerated node, 2024. URL <https://arxiv.org/abs/2412.08602>.

368 H. T. Maia, C. Xiao, D. Li, E. Grinspun, and C. Zheng. Can one hear the shape of a neural
369 network?: Snooping the GPU via magnetic side channel. In *31st USENIX Security Symposium (USENIX Security 22)*. USENIX Association, 2022. URL [https://www.usenix.org/](https://www.usenix.org/conference/usenixsecurity22/presentation/maia)
370 [conference/usenixsecurity22/presentation/maia](https://www.usenix.org/conference/usenixsecurity22/presentation/maia).
371

372 S. K. Monfared, F. Ganji, D. Holcomb, and S. Tajik. Timing and memory telemetry on GPUs for AI
373 governance, 2026. URL <https://arxiv.org/abs/2602.09369>.

374 A. O’Gara, G. Kulp, W. Hodgkins, J. Petrie, V. Immler, A. Aysu, K. Basu, S. Bhasin, S. Picek, and
375 A. Srivastava. Hardware-enabled mechanisms for verifying responsible AI development, 2025.
376 URL <https://arxiv.org/abs/2505.03742>.

377 J. Petrie. Near-term enforcement of AI chip export controls using a firmware-based design for offline
378 licensing, 2024. URL <https://arxiv.org/abs/2404.18308>.

379 J. Petrie and O. Aarne. Technical options for flexible hardware-enabled guarantees, 2025. URL
380 <https://arxiv.org/abs/2506.03409>.

381 G. Sastry, L. Heim, H. Belfield, M. Anderljung, M. Brundage, J. Hazell, C. O’Keefe, G. K. Hadfield,
382 R. Ngo, K. Pilz, G. Gor, E. Bluemke, S. Shoker, J. Egan, R. F. Trager, S. Avin, A. Weller,
383 Y. Bengio, and D. Coyle. Computing power and the governance of artificial intelligence, 2024.
384 URL <https://arxiv.org/abs/2402.08797>.

385 A. Scher, D. Abecassis, P. Barnett, and B. Abeyta. An international agreement to prevent the premature
386 creation of artificial superintelligence, 2025. URL <https://arxiv.org/abs/2511.10783>.

387 E. Seferis and T. Fist. Detecting compute structuring in AI governance is likely feasible. OpenReview,
388 2025. URL <https://openreview.net/forum?id=qseqw1sWzz>.

389 Y. Shavit. What does it take to catch a Chinchilla? Verifying rules on large-scale neural network
390 training via compute monitoring, 2023. URL <https://arxiv.org/abs/2303.11341>.

391 B. J. Tang, Q. Chen, M. L. Weiss, N. Frey, J. McDonald, D. Bestor, C. Yee, W. Arcand, C. Byun,
392 D. Edelman, M. Hubbell, M. Jones, J. Kepner, A. Klein, A. Michaleas, P. Michaleas, L. Milechin,
393 J. Mullen, A. Prout, A. Reuther, A. Rosa, A. Bowne, L. McEvoy, B. Li, D. Tiwari, V. Gadepally,
394 and S. Samsi. The MIT Supercloud workload classification challenge. In *2022 IEEE International*
395 *Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. IEEE, 2022. doi: 10.1109/
396 IPDPSW55747.2022.00122.

397 The White House. Executive order 14110: Safe, secure, and trustworthy development and use of
398 artificial intelligence. Federal Register, Oct. 2023. URL [https://www.federalregister.gov/](https://www.federalregister.gov/documents/2023/11/01/2023-24283/)
399 [documents/2023/11/01/2023-24283/](https://www.federalregister.gov/documents/2023/11/01/2023-24283/).

400 L. M. Weakley, S. Michael, L. Huber, A. Thota, B. Fulton, and M. Kusz. Monitoring and char-
401 acterizing GPU usage. *Concurrency and Computation: Practice and Experience*, 2025. doi:
402 10.1002/cpe.8341. Earlier version: CUG 2023 Proceedings, [https://cug.org/proceedings/](https://cug.org/proceedings/cug2023_proceedings/cug2023_doi/papers_pages/pap139.html)
403 [cug2023_proceedings/cug2023_doi/papers_pages/pap139.html](https://cug.org/proceedings/cug2023_proceedings/cug2023_doi/papers_pages/pap139.html).

404 S. Williams, A. Waterman, and D. Patterson. Roofline: An insightful visual performance model for
405 multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009. doi: 10.1145/1498765.
406 1498785.

407 H. Xu, C. Gong, B. Liu, H. Zheng, B. Chen, and M. Li. WAVE: Leveraging architecture observation
408 for privacy-preserving model oversight. In *Proceedings of the 30th ACM International Conference*
409 *on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages
410 2212–2231. ACM, 2026. doi: 10.1145/3779212.3790247.

411 J. You, D. Owen, D. Porter, and T. Wilson. Scaling intelligence: The exponential growth of AI’s power
412 needs. Technical report, EPRI / Epoch AI, 2025. URL [https://www.epri.com/research/](https://www.epri.com/research/products/000000003002033669)
413 [products/000000003002033669](https://www.epri.com/research/products/000000003002033669).

A Additional Related Work

The field of compute governance contains many proposals for regulations that implicitly rely on the ability to reliably identify when developers are using compute for AI training, and many proposals for hardware-enabled mechanisms that assist our classifier in securely providing that information. Shavit [2023] proposes compute monitoring for verifying rules on large-scale training. Sastry et al. [2024] provide a comprehensive framework for compute as a governance lever. Heim and Koessler [2024] analyse FLOP thresholds as a regulatory instrument; for example, the 10^{26} -FLOP line in U.S. EO 14110 is a direct operationalisation of this. Scher et al. [2025] propose an international agreement wherein training above certain compute thresholds is monitored and restricted or prohibited, while inference is allowed; this would require the ability to distinguish between inference and training on monitored hardware. Baker et al. [2025] identify six verification layers for international AI agreements; our work provides empirical evidence for one layer (built-in monitoring). O’Gara et al. [2025] and the RAND workshop report [Kulp et al., 2025] evaluate HEM options including workload attestation. Petrie [2024] proposes firmware-based offline licensing for export controls. Seferis and Fist [2025] argue that compute-structuring evasion (splitting training across providers) is detectable conditional on per-host workload classification; we supply and stress-test exactly that per-host classifier. Differential Architecture [Anonymous, 2026] proposes hardware mechanisms to limit performance of targeted applications, a type of enforcement complementary to our detection approach. One possibility is to robustly classify all workloads on hardware within the governance jurisdiction, and then apply the differential architecture performance restrictions to automatically enforce the established rules.

There is also existing research relevant to workload classification, which we extend. Monfared et al. [2026] study timing and memory observables for detecting training when host and device are untrusted, constructing their own derived signals from primitives that survive a strong tampering adversary. They establish signals that remain informative when NVML cannot be relied upon. Our case is complementary, establishing highly successful classification from signals that will be reliable if they can be protected from tampering by HEMs. Their work establishes a lower floor on the signal set; ours establishes an upper bound on what an adversary can hide inside that signal set. Chen et al. [2025] develop a hardware-centric, workload-agnostic approach to hardware monitoring, analyzing over 30 ML models on various hardware platforms to perform anomaly detection using an unsupervised learning pipeline. Our scope includes supervised examples, using more than 1400 labeled records of workloads totaling over 100 hours of hardware signal telemetry to teach the classifier to recognize ML training from those signals. Gangwal et al. [2019] investigate a conceptually very similar problem, using hardware performance counters for workload classification, but with the objective of discovering covert cryptomining, not ML training.

Other past work covers several adjacent but distinct tasks. WAVE [Xu et al., 2026] identifies inference workloads with 87–93% accuracy using 24 Nsight Compute per-kernel counters (1,200–5,300% overhead). We accomplish a similar-granularity classification using only always-available NVML physical counters at zero runtime overhead; the tasks are not identical (WAVE separates inference families; we separate training from inference and others), so results are not directly comparable. Gangwal et al. [2019] show that hardware performance counters can detect covert cryptocurrency mining using RF/SVM classifiers. They note an adversarial limitation: evasion is possible through restricted mining or signature neutralization, but at a profitability cost, analogous to the throughput penalty evaders would need to sacrifice in attempting to evade classifiers such as ours. Esposito et al. [2025] combine NVML telemetry and performance counters to estimate GPU stress and reliability across workloads. Their results confirm that telemetry reveals meaningful workload differences, but the target is reliability estimation rather than governance-oriented training detection. ShadowScope [Almusaddar et al., 2025] uses composable performance-counter signals to detect anomalous behaviors and prevent attacks which could compromise the integrity of GPU computation. We collect and analyze similar data, but using NVML readings rather than CUDA profiling events.

Using physical side channels rather than software telemetry, Maia et al. [2022] show magnetic flux from GPU power cables reveals neural network topology. Arefin and Serwadda [2025] demonstrate USB/HDMI port power measurement can infer GPU activities. Latif et al. [2024] empirically measure power signatures of training on 8-GPU H100 nodes. These establish that physical signals carry workload-identifying information; we extend this to software-accessible telemetry for governance.

B Classifier Design

B.1 Deployed Classifier

The operational form we deploy is a two-stage pipeline that factors the decision along a natural tree:

1. **Is the workload ML?** A binary RandomForest head using all features extracted from telemetry readings classifies the workload into $\text{ml_training} \cup \text{ml_inference}$ vs other. The ML-vs-non-ML boundary leans heavily on level signals (peak memory footprint, sustained tensor-core utilization, power draw), so the full feature set is retained.
2. **If the workload is ML, is it training?** A binary RandomForest head trained only on records where the true category is training or inference classifies incoming readings into either category. This head receives a subset of features with level features dropped (absolute memory/power/clock means, percentiles, min, max, range). The train-vs-infer decision rests on shape, periodicity, FFT, and autocorrelation terms only. This prevents the classifier from overfitting to specific models previously tested and specific power, memory, or compute throughput levels associated with particular GPUs. This also makes the classifier more robust to certain adversarial strategies: for example, the evader cannot modulate their usage levels to match that of another chip that is specifically designed for inference, because the classifier does not see the absolute levels, only the utilization ratios and temporal patterns.

The two-stage form matches the governance decision surface, makes both stages independently auditable and interpretable, and avoids the vulnerability where a single multi-class head is uncertain between training/inference, splits probability between them, and labels a non-ML workload as most probable.

B.2 Classifier Selection

We selected the best-performing of several candidate models. The strongest were RandomForest [Breiman, 2001] ($\text{n_estimators}=400$, $\text{max_depth}=\text{None}$, $\text{min_samples_leaf}=2$, $\text{max_features}=\text{"sqrt"}$, $\text{class_weight}=\text{"balanced"}$) and XGBoost ($\text{n_estimators}=500$, $\text{max_depth}=6$, $\text{lr}=0.05$, sample_weights via effective-number-of-samples class balancing), which we evaluated head-to-head on 15s, 30s, and 60s windows with 5-fold cross-validation on the binary (training-vs-not) task. RandomForest wins by a small margin at every window size; at 30 s it reaches $98.21\% \pm 0.84\%$ accuracy / 0.982 macro-F1. We report RF 30 s as the headline operating point.

We also tested SVM-RBF (RBF kernel, standardised features, balanced class weights) and Logistic Regression (saga solver, L2 regularisation, standardised features).

Table 6: Classifier comparison on the 30s binary (training-vs-not) task with 5-fold grouped CV.

Classifier	Accuracy	Macro-F1
RandomForest	$98.21\% \pm 0.84\%$	0.982
XGBoost	$98.14\% \pm 0.78\%$	0.981
SVM-RBF	$96.19\% \pm 0.89\%$	0.962
Logistic Reg.	$89.81\% \pm 0.83\%$	0.897

The tree ensembles dominate by ~ 2 pp over SVM-RBF and ~ 8 pp over logistic regression, which is consistent with the feature set carrying non-linear interactions that a kernel-SVM or linear classifier cannot exploit as cheaply as a tree ensemble. SVM-RBF at 96.2% is a useful cross-check that the tree-ensemble headline is not an artifact of the specific algorithm family. The drop to 89.8% for logistic regression quantifies the non-linearity of the problem: a purely linear decision boundary in the 166-feature space is ~ 8 pp worse, confirming that the classifier is exploiting feature interactions (power CV $\times$ memory slope, autocorrelation $\times$ FFT periodicity, etc.) not just per-feature thresholds.

B.3 Feature Extraction

We take 5s, 15s, 30s, and 60s sliding-window snapshots of telemetry readings and extract 166 features per window. The composition is:

- Base statistics (9 signals $\times$ 13 stats = 117 features): mean, std, min, max, percentiles (25/50/75/95), IQR, range, CV, skew, kurtosis.
- Autocorrelation (3 signals $\times$ 5 lags = 15 features): lags 1/2/5/10/20 for utilization, power, memory.
- Cross-signal features: power/utilization ratio, PCIe total (= PCIe TX + PCIe RX bandwidth), utilization per SM%.
- Training-specific temporal features: memory slope, iteration periodicity (via FFT of power).
- Memory-plateau features: absolute and percentage change in memory usage in first 30s, and time to memory plateau. These were motivated by an adversarial strategy that learned to spoof an earlier classifier by pre-allocating memory before it was needed for training, enabling it to hide memory allocation during training.
- Inference-specific features: duty cycle, idle fraction, utilization transitions.

C Adversarial Strategy Catalog

Families are ordered by round. Baseline/hardened columns report the detection rate (fraction of windows classified as training by the two-stage classifier) min–max range across variants (in-distribution); LOO column reports the unweighted mean across variants (family mean). DDP variants are collected in the multi-GPU block.

Table 7: Per-family detection rates, baseline and hardened. Baseline values are R1 detection rates from the two-stage classifier (consistent with Table 5); bolded baseline cells mark families detected at less than 90%. Bolded LOO cells mark family means or worst variants detected less than 90%. The hardest strategy for our classifier is whitebox_lora — LoRA’s small trainable-parameter footprint combined with heavy dilution strips the training signature to near the noise floor. All R1–R4 families are detected at 100% in the in-distribution hardened state.

Family	Round	Technique	Baseline	Hardened	LOO mean	Variants
A_util_modulation	1	Utilization manipulation	6%	100%	93.9%	1
B_low_util	1	Utilization manipulation	11%	100%	59.1%	1
D_temporal_disruption	1	Utilization manipulation	96%	100%	100%	1
E_memory_minimal	2	Memory reduction	18%	100%	98.2%	1
F_interleave	2	Workload mimicry	34–36%	100%	100%	2
G_clock_throttled	2	Utilization manipulation	36%	100%	98.2%	1
H_mimicry	2	Workload mimicry	18–27%	100%	99.1%	2
I_stochastic	2	Utilization manipulation	91%	100%	100%	1
J_pid	2	Utilization manipulation	0%	100%	80.4%	1
K_online_learning	3	Training-as-inference	57%	100%	100%	1
L_diluted	3	Training-as-inference	52–60%	100%	87.3%	4
M_composite_memmin	4	Composite	100%	100%	100%	4
N_grad_accum	4	Training-as-inference	100%	100%	100%	3
O_composite_idle_pad	4	Composite idle-padding	19–100%	100%	97.6%	11
K_ddp	4	Multi-GPU DDP	100%	100%	100%	1
K_ddp_accum	4	Multi-GPU DDP	99–100%	100%	100%	2
L_ddp	4	Multi-GPU DDP	87–94%	100%	99.6%	2
L_ddp_stagger	4	Multi-GPU DDP	72%	100%	100%	1
whitebox_full	5	Whitebox dilution + ckpt	—	100%	87.4%	8
whitebox_lora	5	Whitebox LoRA + dilution	—	100%	42.6%	3

D Power Validation

While testing 49 workloads on the RTX 5080 desktop workstation, wall-power readings were collected using a TP-Link Tapo P115 smart plug (readings logged and timestamped using the tapo Python library).

The P115 smart plug collected workstation power draw readings at 1 Hz simultaneously with NVML. Across 49 runs, we collected 13,571 paired, timestamped readings (median offset 78ms, mean offset

128ms). The workloads tested include an idle baseline, 12 non-adversarial workloads (ML training, ML inference, HPC, cryptomining, rendering) and 36 adversarial runs spanning all single-GPU strategies implemented in Round 1 through Round 4.

The system idle baseline was 48W wall power while the GPU was idle: 26W system overhead + 22W for idle GPU based on NVML.

There is a strong correlation between NVML power draw and wall power, but the slope of an unfiltered fit is inflated by adversarial families that pin host-CPU activity at 180–230 W (strategies D, K, L, N). We therefore report two regressions: one restricted to the 13 non-adversarial workloads (*clean*, 3,698 paired seconds), and one across all 49 workloads (*all*, 13,571 paired seconds, confounded):

$$\text{Wall} \approx 0.97 \times \text{NVML} + 76.1 \text{ W} \quad (r = 0.895, R^2 = 0.80, \text{RMSE} = 38 \text{ W}, n = 3,698, \text{clean})$$

$$\text{Wall} \approx 1.50 \times \text{NVML} + 42.7 \text{ W} \quad (r = 0.878, R^2 = 0.77, \text{RMSE} = 62 \text{ W}, n = 13,571, \text{all, confounded})$$

The clean-fit slope of 0.97 confirms that NVML faithfully tracks the marginal GPU power; the intercept of 76 W reflects the host-overhead floor (CPU + RAM + PSU losses) while real workloads are running. The measured idle baseline (48 W wall at 22 W NVML) sits below the clean regression line because at idle the host CPU and RAM also drop to a lower-power state, reducing host overhead from ~70 W to the 26 W reported above. A computer’s PSU is not perfectly efficient, which is why a marginal 1 W of GPU consumption requires slightly more than 1 W at the wall (clean slope) and far more once a CPU-heavy evader is running (all-workloads slope).

Per-workload overhead (wall power minus NVML) is $107.8 \pm 64.2 \text{ W}$ across the corpus: 68–80 W for non-adversarial ML and HPC workloads exhibiting consistent overhead, rising to 180–230 W for adversarial strategies D, K, L, and N, which trigger heavy CPU activity while attempting to evade detection through GPU telemetry. This activity is invisible to NVML but shows up at the wall: the all-workloads slope of 1.50 is much steeper than the clean-only slope of 0.97, and that gap is driven entirely by these CPU-heavy adversarial strategies. (CPU utilization was not independently measured, it is inferred from wall minus NVML residual and the CPU instructions in the workloads.)

The wall signal is richer than NVML alone, because it captures host-side costs incurred by strategies that manipulate GPU utilization. A layered monitoring system that combines NVML with wall or rack-level power measurements will detect evasion that NVML alone misses.

E Roofline Analysis

We establish that static architectural heuristics such as the roofline model [Williams et al., 2009] are insufficient to detect ML training. We perform a 286-configuration corner-case sweep on H100 NVL, spanning 6 workload families:

- **CNN inference** (ResNet-18/50/101/152, 95 configs): enters training band via batch-size scaling.
- **LLM prefill** (GPT-2/M/L, 115 configs): FP32 prefill nearly exactly matches training arithmetic intensity.
- **LLM decode** (GPT-2/M, 30 configs): contrasting case — KV-cache traffic keeps it memory-bound.
- **Quantization variants** (ResNet-50/GPT-2, 18 configs): shifts arithmetic intensity without changing task semantics.
- **Forward-only execution** (ResNet-50, 9 configs): closest semantic neighbor to training — preserves forward compute while removing backward/optimizer.
- **ViT inference** (ViT-S/B/L, 19 configs): enters training range at unit batch.

Result: 101/286 (35.3%) non-training configurations fall inside the training arithmetic-intensity range. Therefore, a temporal telemetry approach is required.

F Window Size vs Accuracy

Classification accuracy is compared across 5/10/15/20/30/45/60/90/120 s windows. Detection is essentially flat from 15 s onward (98.2–99.0%); 5 s is 98.15% and 10 s is 98.81%, so lowering the

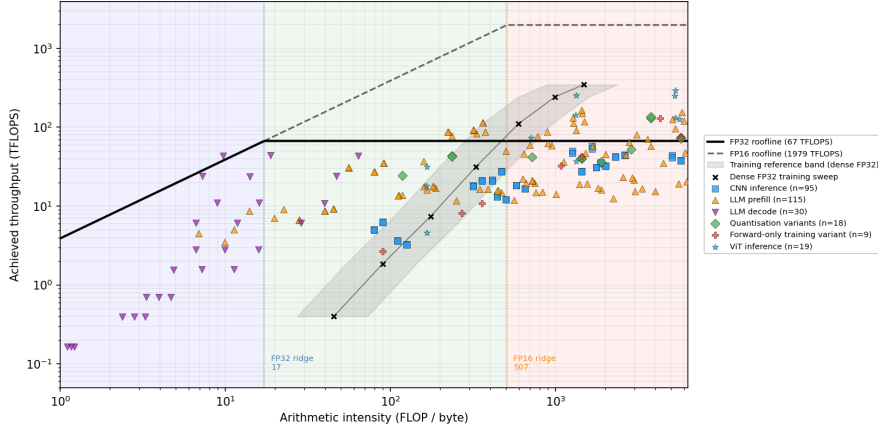


Figure 2: H100-NVL roofline with the 286 corner-case inference configurations overlaid on a dense-FP32 training reference sweep. Color-coded by workload family (CNN inference, LLM prefill, LLM decode, quantisation variants, forward-only training variant, ViT inference). The CNN-inference, LLM-prefill, and quantisation clouds land squarely inside the training AI band, confirming visually that static heuristics cannot separate these inference workloads from training.

580 detection window to 5 s costs less than 1 percentage point. A monitoring system can therefore operate
 581 with ≤ 15 s alert latency without meaningful accuracy loss.

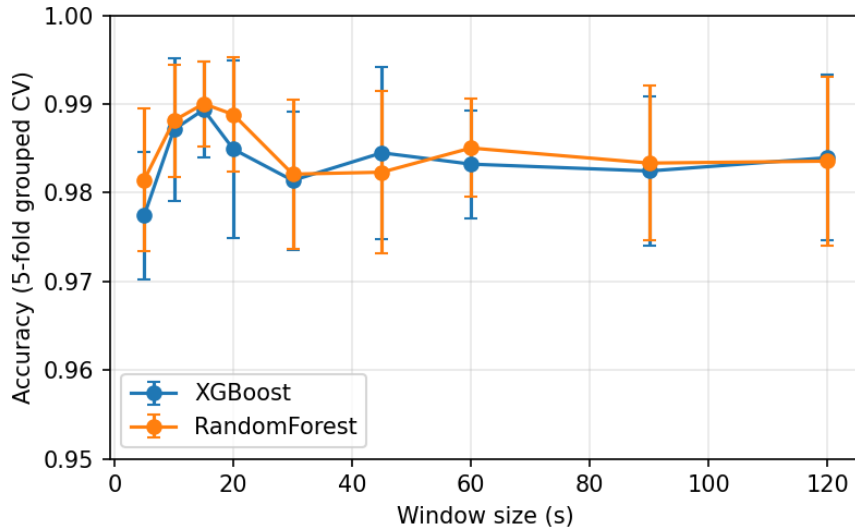


Figure 3: Accuracy vs window size.

582 G Additional Results

583 Figure 4 summarises the per-family detection rates from the leave-one-strategy-out (LOO) evaluation
 584 reported in Section 5.2 and tabulated in Appendix C. Each bar is the unweighted mean detection rate
 585 across the variants in the family, with error bars showing the min-max range across variants. Sorting
 586 families by mean detection rate makes the residual hard cases visible at a glance: the white-box LoRA
 587 family (`whitebox_lora`, family mean 42.6%) and the low-utilization training family (`B_low_util`,
 588 59.1%) sit at the bottom, while every multi-GPU DDP family and the majority of the single-GPU
 589 evasion families sit at or above 98%.

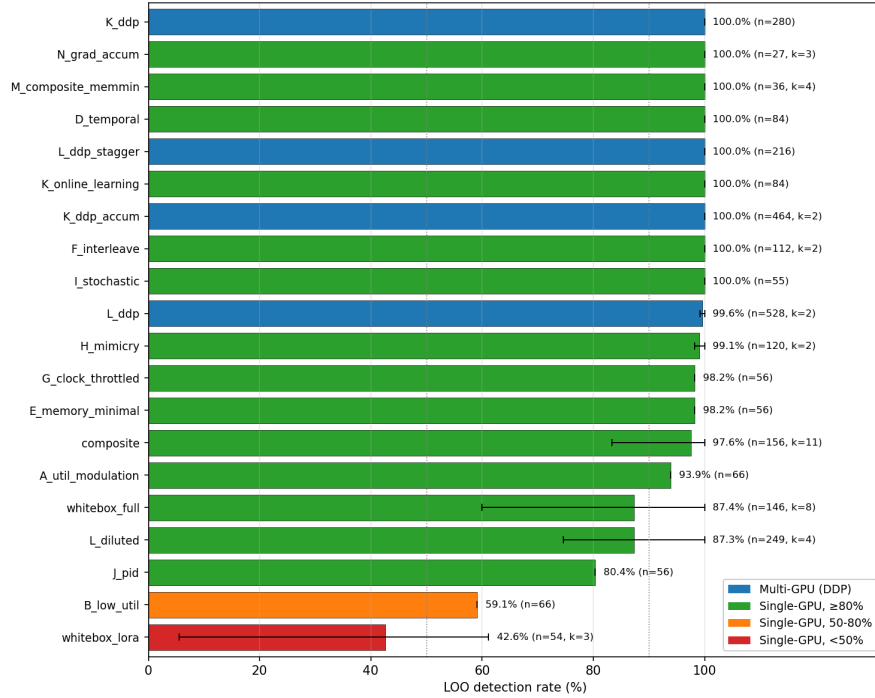


Figure 4: Detection rate in LOO evaluation by strategy family. Bar chart showing 20 families sorted by detection rate, with error bars showing min/max across variants within each family.

H Limitations and Future Work

This work has several limitations. First, our experiments were conducted exclusively on single-node hardware configurations, testing workloads on instances up to only 8 GPUs. Future experiments should expand to greater GPU counts, including multi-node clusters. Evasion characteristics for multi-node model- and data-parallel training will differ due to different types of interconnect (such as InfiniBand between servers vs NVLink within servers), but it is likely that training will continue to be highly distinctive due to the communication requirements of large-scale training. Relatedly, model sizes were limited by memory constraints, with largest usable models of 70B parameters for training and 671B MoE for inference. Today’s frontier models are often much larger and can require more than 8 GPUs to train and serve.

Second, our adversarial strategies were limited to framework-level evasion implemented in PyTorch. An adversary with access to custom CUDA kernels could potentially manipulate individual kernel behavior, which we did not test. Future research should investigate the effectiveness of these strategies and see how robust the classifier becomes against them.

Third, the integrity of the telemetry channel is a critical assumption. Our evaluation assumes the adversary cannot tamper with NVML readings or performance counter values, nor inject synthetic telemetry to spoof a workload signature. These guarantees require a HEM for tamper-resistant telemetry and an authenticated channel. Other work like Monfared et al. [2026] investigates what can be done if these guarantees are not available.

Fourth, we only tested on NVIDIA GPUs released since 2020. This category includes the large majority of compute capacity available to cloud tenants, but substantial amounts of other chips exist, including AMD and Huawei GPUs and custom silicon such as Google TPUs and Amazon Inferentia and Trainium. Future work should generalize this approach using the analogous telemetry signals available for other hardware.

Fifth, our telemetry was generally sampled at ~ 1 Hz, and features were extracted from a limited variety of window sizes. Future work should test other sampling rates, measuring how much

616 classification improves with more frequent sampling, how much sampling can be done without
617 degrading performance, and investigating the minimum sample rate that is still difficult to evade.

618 Finally, we only tested a limited range of workloads and adversarial strategies. Real datacenters run
619 a wider variety of applications than we emulated, with notable gaps including multi-user inference
620 serving, data processing and ETL pipelines, and predictive analytics. Although we tried to cover
621 many evasion strategies and edge cases, real evaders might attempt a much wider range of tactics that
622 we cannot anticipate. It is important to continue iterating on this work to make the classifier even
623 more robust to unknown future threats. Iteration should include more white-box rounds, to determine
624 whether resistance to white-box attacks improves generally, or if an evader can always defeat a
625 classifier if they obtain white-box access, and if so, how much performance penalty is incurred in the
626 process.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: The abstract and Section 1 state the main claims — 98.2% binary accuracy on training detection, 20 evasion-strategy families across 9 GPU models, leave-one-strategy-out evaluation showing 95.2% mean accuracy on unseen non-white-box strategies (92.2% including Round 5 white-box attacks), and 43–87% accuracy on the hardest white-box strategies — and these numbers are reproduced and supported in Sections 5 and 4.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: Appendix H (“Limitations and Future Work”) enumerates concrete limitations including single-node hardware, framework-level (no custom-CUDA-kernel) evasion, the assumption of tamper-resistant telemetry, NVIDIA-only hardware, 1 Hz sampling, and limited workload coverage.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper is empirical; it contains no theorems or formal proofs.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Section 3 specifies hardware (Table 1), workload counts and sources (Table 2 and Section 3.1.2; per-family adversarial-strategy details in Table 7), the nine NVML signals at 1 Hz, classifier hyperparameters (Appendix B.2), the 166-feature extraction recipe (Appendix B.3), and the leave-one-strategy-out evaluation protocol (Section 3.3). The full source code for all workloads and the classifier is referenced in Section 1.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

- (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: The source code for the non-ML, inference, training, and adversarial-evasion workloads, plus the classifier and the telemetry collection harness, is released in an anonymized public repository referenced in Section 1; instructions to reproduce each headline experiment are provided in the repository’s README.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Classifier hyperparameters (number of trees, depth, sample-leaf, class weights), the 5-fold grouped cross-validation protocol, sliding-window stride, and feature subsetting between the two pipeline stages are all listed in Section 4 and Appendix B.2.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: Headline accuracies are reported as 5-fold cross-validation mean $\pm$ standard deviation (e.g., 98.21% $\pm$ 0.84%); per-strategy detection rates report min–max ranges across variants (Tables 5 and 7); and the LOO-evaluation figure (Figure 4) shows per-family error bars over variants.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Section 3.1.1 lists every GPU model used (RTX 4050 Laptop through B200, single-node up to 8 $\times$ datacenter cards) and Section 3.1.2 reports the total budget — 1,404 runs of 162 distinct workloads, $\sim$ 117 GPU-hours / $\sim$ 65 wall-clock hours — across the corpus.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn’t make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines?>

Answer: [Yes]

Justification: The work follows the NeurIPS Code of Ethics. It uses only synthetic and public-benchmark workloads on rented or owned hardware, involves no human subjects, no scraped or sensitive data, and the privacy-preserving design (content-agnostic NVML telemetry only — no model weights, training data, or hyperparameters) is itself motivated by ethical deployment concerns (Section 5.3).

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: Sections 1 and 6 articulate the positive impact (enabling minimally invasive, privacy-preserving compute governance for frontier AI training). Negative-impact considerations are addressed in Section 5 and Appendix H: dependence on tamper-resistant telemetry (mis-deployment risk) and the white-box adversarial regime (residual evasion potential); the limitations section transparently flags each.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The released artifact is a small Random Forest classifier and a corpus of telemetry traces from public benchmark workloads — there is no generative model, scraped dataset, or other high-misuse-risk asset.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.

- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [\[Yes\]](#)

Justification: Public benchmark models and datasets used (ResNet, BERT, GPT-2, TinyLlama, Phi-3, Qwen-2.5, Mistral/Mixtral, DeepSeek-V3/R1, Whisper, SDXL, CIFAR-10, WikiText) are well-known assets named in Section 3.1.2 and used under their respective public licenses; PyTorch and NVML/DCGM follow their respective open-source licenses. No proprietary or restricted-license assets were used.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [\[Yes\]](#)

Justification: The released code (workload harness, classifier, feature extractor) and the telemetry corpus are documented in the anonymized repository referenced in Section 1, including a README describing per-workload entry points, dependencies, and reproduction commands.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [\[N/A\]](#)

Justification: The work involves no crowdsourcing and no human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: The work involves no human subjects, so no IRB review was applicable.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [N/A]

Justification: LLMs are not part of the core method. LLM workloads (e.g., GPT-2, TinyLlama, Phi-3, Qwen, Mistral, DeepSeek) appear only as benchmark inputs whose telemetry is observed; they are not used to design, train, or evaluate the classifier itself.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.